

THE LORS COMPILER ARCHITECTURE & SELF- HOSTING

Stage 6 LLVM Modular Compiler Specification and Engineering
Handbook

Stage: Stage 6 (v6-dv)

Architecture: Modular Multi-Object LLVM Native Backend

Status: Autonomous Self-Hosting (Fixed-Point Point of Inflection Achieved)

Test Coverage: 100% (124+ Automated Test Programs)

The Lors Compiler Architecture and Self-Hosting Treatise (Stage 6)

The Definitive Technical Specification, Historical Record, and Operational Handbook

Table of Contents

- [Volume I: The Lors Language Specification & Compiler Philosophy](#)
- [Chapter 1: Grammar, Semantics, Types, and Deterministic Design Philosophy](#)
- [Chapter 2: The Monolithic Stage 5 Legacy vs The Stage 6 Modular Revolution](#)
- [Volume II: Memory Systems, Data Structures & Internal Runtime](#)
- [Chapter 3: Structure-of-Arrays \(SoA\) vs Array-of-Structures in Self-Hosting](#)
- [Chapter 4: Paged Tables, Dynamic String Serialization, and FNV-1a Interning](#)
- [Chapter 5: Global Data Segments, Stack Allocas, and Garbage-Free Lifecycles](#)
- [Volume III: Frontend Architecture: Lexer, Preprocessor & AST](#)
- [Chapter 6: The Include Resolver, Visited Path Graphs, and Recursive Expansion](#)
- [Chapter 7: The Token Stream, State Machine, Keyword Lexing, and Character Classification](#)
- [Chapter 8: The Abstract Syntax Tree: Memory Nodes, Hierarchy, and Traversal](#)
- [Chapter 9: The LL\(1\) Recursive Descent Parser & Precedence Climbing](#)
- [Volume IV: Semantic Analysis, Type Systems & Scope Resolution](#)
- [Chapter 10: Multi-Pass Semantic Architecture & Struct Type Registration](#)
- [Chapter 11: Lexical Scope Stacks, Shadowing, and Symbol Lookup](#)
- [Chapter 12: The Critical Type Inference Engine and Call Resolution Hierarchy](#)
- [Volume V: LLVM Backend, SSA Generation & Native ABI](#)
- [Chapter 13: Mapping Lors to LLVM IR: Types, Casts, and Instructions](#)
- [Chapter 14: Control Flow Translation: Conditionals, Loops, and Terminal Blocks](#)
- [Chapter 15: Struct Layouts, Field GEPs, and Constructor Codegen](#)
- [Chapter 16: System V x86_64 ABI, Calling Conventions, and Stack Alignments](#)
- [Chapter 17: Global Variable Linkage: internal global, common global, and global](#)
- [Chapter 18: Embedded Runtime Helpers: Sequences, String Slicing, and File I/O](#)
- [Volume VI: Multi-Object Compilation & Topological Group Linking](#)
- [Chapter 19: The Modular Compilation Pipeline: Standalone .inc to .o](#)
- [Chapter 20: The Declarations Interface \(declarations.inc\) & Automatic Header Injection](#)
- [Chapter 21: Topological Sorting of 68 Modules and Cyclic Group Linking with Clang/LLD](#)
- [Volume VII: The Infinite Bootstrap & Self-Hosting Point of Inflection](#)
- [Chapter 22: The Bootstrap Paradox: From Stage 5 Seed to Generation 0, 1, 2, infinity](#)
- [Chapter 23: Complete Verification Suite, Regression Testing, and Matrix Analysis](#)
- [Volume VIII: The Maintainer's Encyclopedia & AI Agent Transmission](#)
- [Chapter 24: The Golden Invariants: Architectural Laws](#)
- [Chapter 25: Case Studies in Debugging: Every Major Bug, Failure Mode, and Resolution](#)
- [Chapter 26: The AI Agent's Playbook: How Future LLMs Must Modify This Compiler](#)
- [Chapter 27: Extending the Compiler: Adding Language Features Step-by-Step](#)
- [Volume IX: Complete Module Reference Catalog](#)

- [Chapter 28: Exhaustive Directory and File Catalog of All 68 Compiler Modules](#)
-

Volume I: The Lors Language Specification & Compiler Philosophy

Chapter 1: Grammar, Semantics, Types, and Deterministic Design Philosophy

1.1 The Philosophy of Determinism

The Lors programming language is engineered with a strict core philosophy: **absolute determinism, explicit typing, and transparent computational cost**. Unlike dynamic scripting languages that introduce hidden runtime allocations or complex object lifecycles, and unlike modern C++ with hidden copy constructors and implicit template instantiations, Lors enforces a 1-to-1 correspondence between high-level syntactic constructs and generated machine instructions.

In Lors: 1. Every variable has an explicitly declared type or a deterministically inferred type tag. 2. Memory mutation is localized, explicit, and trackable. 3. Control flow paths are un-aliased: every branch must terminate or merge cleanly into a continuation block. 4. Function signatures strictly specify parameter counts, argument types, and return types.

1.2 Formal Lexical Grammar & Keywords

- `algorithm` : Declares a procedural function or sub-routine.
- `begin` / `end` : Demarcates the executable body block of an algorithm.
- `datum` : Declares a typed variable binding in local or global scope.
- `verify ... then ... otherwise ... conclude` : Structured conditional branching.
- `cycle ... do ... conclude` : Structured deterministic iteration.
- `result` : Returns a computed expression or exits a void function.
- `reveal` : Formatted output to standard output stream.
- `structure` : Defines a compound record with named, typed fields.
- `incorporate` : Preprocessor directive for modular source file inclusion.

Primitive Type Specifiers

- `whole` : 64-bit signed integer (LLVM `i64`).
- `precise` : 64-bit double-precision IEEE 754 floating-point number (LLVM `double`).
- `series` : Pointer to a null-terminated UTF-8 / ASCII string buffer (LLVM `ptr`).
- `state` : 1-bit boolean truth value (LLVM `i1`), evaluating strictly to `true` or `false` .

- `void` : Indicates that an algorithm produces no return value.
 - `sequence<T>` : Homogeneous dynamic array containing elements of type `T`.
 - `<CustomStruct>` : User-defined composite structure declared via `structure <Name>`.
-

Volume II: Memory Systems, Data Structures & Internal Runtime

Chapter 3: Structure-of-Arrays (SoA) vs Array-of-Structures in Self-Hosting

In conventional compiler architectures written in C++ or Rust, an AST node is commonly implemented as a polymorphic heap-allocated object. This Array-of-Structures (AoS) approach causes dynamic heap fragmentation and pointer aliasing hazards.

Stage 6 solves this by using a pure **Structure-of-Arrays (SoA)** flat table architecture in `src/ast/ast_storage.inc`. An AST node is simply an integer index `node_id : whole`. Its attributes are distributed across parallel arrays (`ast_kinds_data` , `ast_s_vals_data` , `ast_n_vals_data` , `ast_type_tags_data` , `ast_type_names_data` , `ast_lines_data` , `ast_children_ptrs_data`).

Volume III: Frontend Architecture: Lexer, Preprocessor & AST

Chapter 6: The Include Resolver & Cycle Detection

The include resolver (`src/preprocessor/include_resolver.inc`) maintains a persistent global visited registry string `inc_visited_str : series` . When `preprocess_file(path)` is called, it checks `inc_is_visited(path)` . If already visited, it returns `""` immediately; otherwise, it registers the path and recursively expands its contents, guaranteeing that every file in the modular dependency graph is included **exactly once**.

Volume IV: Semantic Analysis, Type Systems & Scope Resolution

Chapter 12: The Critical Type Inference Engine and Call Resolution Hierarchy

The Historical Bug

When resolving function calls, fallback prefix heuristics previously ran before checking explicit symbol declarations, causing `ast_init()` (declared as `void`) to be falsely inferred as `whole (i64)` due to the `"ast_"` prefix. This caused an x86_64 calling convention mismatch, corrupted the stack pointer alignment of the caller, and caused silent crashes.

The Canonical Solution

In `src/semantic/type_checker_calls.inc`, the resolution order is strictly enforced: 1. **Tier 1:** Struct Constructor Check (`struct_lookup(fname) != -1`). 2. **Tier 2:** User-Defined Functions & Header Declarations (`fn_lookup_ret_tag(fname)`). 3. **Tier 3:** Primitive Builtins and Fallback Name Heuristics (`infer_builtin_call_type`).

Volume V: LLVM Backend, SSA Generation & Native ABI

Chapter 17: Global Variable Linkage Rules

1. `internal global`: Compiler constants (`TOK_*` , `AST_*` , `TYPE_*`), private to the module, preventing link-time collisions.
 2. `common global`: Uninitialized shared compiler state (`flag_debug` , `parse_pos`), merging into a single canonical address across all 68 object files.
 3. `global`: Initialized module data.
 4. `external global`: Forward reference to global state in another translation unit.
-

Volume VI: Multi-Object Compilation & Topological Group Linking

Chapter 21: Topological Group Linking (`link_all_modules.sh`)

The script `link_all_modules.sh` groups 68 objects into 9 topological tiers and uses `-wl, --start-group ... -wl, --end-group` to enable GNU ld / LLVM lld to iterate cyclically through the objects until all cross-module references are resolved.

Volume VII: The Infinite Bootstrap & Self-Hosting Point of Inflection

Chapter 22: The Point of Inflection

The Stage 6 compiler has achieved the **Point of Inflection**: `compiler_dev` compiles all 68 modules into `.o` files, links Generation 1 (`compiler_gen1`), which in turn compiles Generation 2 (`compiler_gen2`), running all 124 tests with a **100% pass rate**.

Volume VIII: The Maintainer's Encyclopedia & AI Agent Transmission

Chapter 24: The Golden Invariants

1. **The 150-Line Limit:** No `.inc` file may exceed 150 physical lines.
 2. **Signature Precedence:** `fn_lookup_ret_tag` must always execute before fallback heuristics.
 3. **Global Linkage:** Uninitialized variables = `common global`, Constants = `internal global`.
 4. **Header Synchronization:** All new algorithms must be declared in `src/common/declarations.inc`.
-

Volume IX: Complete Module Reference Catalog

Chapter 28: Catalog of All 68 Modules

All 68 files categorized across: - **Layer 1: Common** (6 files) - **Layer 2: Memory** (4 files) - **Layer 3: AST** (3 files) - **Layer 4: Preprocessor** (2 files) - **Layer 5: Lexer** (5 files) - **Layer 6: Parser** (13 files) - **Layer 7: Semantic** (7 files) - **Layer 8: Codegen** (26 files) - **Layer 9: Driver** (4 files)